A Tour of Cryptography

Austin D. Richards

COINS Senior Project Fall 2007

Dear Reader,

I wish to thank you for taking the time to read over my senior project. I hope that this come across as an understandable and informative explanation to the world of encryption. This paper is the accumulation of approximately 6 years of academic training – earning an associates from Trident Technical College and ending (for now) in a Bachelor's from Charleston Southern University.

Along the way there were people who I could not have gotten this far with out:

Jamie, my wife – Thank you for all your support and confidence. Having you by my side has reminded me that anything is possible, and to not quit. I love you with all my heart <3

My Parents – Whose love and raising has helped me grow into the person I am now. I know I can always count on you two. I love you guys!

Gary Underwood – For helping me grow into the best software developer I could throughout my last two years of undergraduate study. And for being a royal pain, in all your classes – sincerely Thank you.

Mr. Roberts – For giving me the opportunity to be a lab assistant and assist others. Teaching is genuine indicator of understanding, thank you for allowing me to express my understanding.

Dr. Sessions and Dr. Lin - I know I only had a couple of classes with each of you, but your support and confidence will carry through with me into my career and possibly Grad. School.

All my past teachers – For pushing me to always do my best and keep me from slacking off.

Table of Contents

Block Ciphers Overview

Block Ciphers allow the output result to be approximately the same size as the message, rounded up (like a ceiling function) to the nearest whole block (see the section on padding). The current trend in block sizes tend to be about 64 bits, with the forecast of increasing this to 128 bits (RSA Labs 2.1.4). Block ciphers tend to operate at a reasonable speed, yet offer reasonable protection at the same time.

Block ciphers are used as fundamental building blocks for common applications including: pseudorandom number generators, stream ciphers, and hash functions (Menezes 223). Block ciphers are ciphers that require a predetermined block size and the encryption function depends on that size[1]. Therefore they require data to be handled in multiples of block size *n*.

Given that encryption's goal is to hide as much information as possible, block ciphers inherently give away some information. Assuming that a message of arbitrary length is encrypted using a block cipher, padding must occur. Padding usually occurs at the end of the file and has some predictable value. As it takes a minimum of 7 bits to represent 64 numbers, a byte is the

---

1 The Handbook of Applied Cryptography page 224 formally defines a block cipher as such:

A block cipher is a function .. which maps *n*-bit plaintext blocks to *n*-bit cipher-text blocks; *n* is called the *blocklength*. It may be viewed as a simple substitution cipher with large character size. The function is parameterized by a *k*-bit key K, taking values from a subset *K* (the *key space*) of the set of all *k*-bit vectors V_k. It is generally assumed that the key is chosen at random. Use of plaintext and ciphertext blocks of equal size avoids data expansion.

To allow unique decryption, the encryption function must be one-to-one (i.e., invertible). For *n*-bit plaintext and ciphertext blocks and a fixed key, the encryption function is a bijection. The number of keys is $|K|$, and the *effective key size* is lg $|K|$, an; this equals the key length if all *k*-bit vectors are valid keys ($K = V_k$). If keys are equiprobable and each defines a different bijection, the *entropy* of the key space is also lg $|K|$.

smallest common unit that can hold 64 values, therefore the last byte of the file stands a good chance of being an encrypted 0-8. If this last number can be derived, then every tail number of each block an be derived. Depending on the padding algorithm used, if this last byte is *n*, then *n* bytes can be possibly be unraveled.

Padding and Cipher Modes

To make messages fit a multiple of block size *n,* various padding methods have been standardized. A document by the National Institute of Standards and Technology outlines some of these standards and also comments, that it is possible to not need padding if the size can be inferred or indicated within the document (United States Department of Commerce 17) . Hiding the amount of padded bytes sounds to be more secure, as no part of the document can be implied.

NIST mentions five modes of operation: Electronic Codebook mode (the one utilized in my code), Cipher Block Chaining Mode, Cipher Feedback Mode, Output Feedback Mode, and Counter Mode. The electronic codebook method (ECB) is the simplest of the five, however it is also the least secure, as the encryption and message are independent. The other four modes use the message (or it's ciphered version) to assist in the next block to encrypt (United States Department of Commerce 9 -17).

The electronic codebook method (ECB) takes the plaintext message precisely as given and runs the encryption algorithm as prescribed. It does no attempt to further conceal the message through it's processing of the plaintext. The resulting ciphertext will therefore be within

one block length of the original message (United States Department of Commerce 9-10).

The cipher block chaining mode (CBC) takes a more elaborate method to encrypt the original data. After a block is encrypted it is then XORed with the next block, therefore each block of plaintext to be encrypted = a block of the message XOR the previous ciphered block. To initiate this process, the first message block is XORed with a predetermined initialization vector[2]. The resulting ciphertext will therefore be within one block length of the original message (United States Department of Commerce 10-11).

The cipher feedback mode (CFB) uses an initialization vector like the CBC, but utilizes it in different fashion. CFB starts by encrypting the initialization vector, it then takes a predetermined number of bits *s* and XORs *s* bits of the message with the first *s* bits of the produced ciphertext. The ciphered bits of the current process are then concatenated with the previous plaintext discarded bits. This new block is ciphered and the XORed with the message, and the process continues (United States Department of Commerce 11-13).

The output feedback mode (OFB) is similar to the CFB without any discarding. The process is started with an initialization vector being encrypted. This encrypted input is XORed with the first block of the plaintext message. The ciphered initialization vector is then encrypted

---

2 The initialization vectors used here are described in the SP 800-38A, published by the NIST. It states the following facts about the vectors:

- Does not need to be secret, therefore can be sent with the ciphered data
- CBC and CFB require unpredictable initialization vectors
  - Use the shared key to encrypt some nonce OR
  - Random data block (using the same seed and generator on each side)
- OFB does does not require unpredictability, but it does require uniqueness or it may be compromised

and XORed with the next block of the plaintext message (United States Department of Commerce 13-15).

The counter mode (CTR) takes some of the simplicity of ECB with the variety of the others to try to achieve a median. The output of each block of CTR does not affect the future block, instead a counter[3] is used. The counter is encrypted, and the output of that encryption is then XORed with the output bytes of the message. This seems to give the further advantage of the resulting output can be the same size as the message and therefore ignore any padding issues, since the XOR will automatically add the extra bits (United States Department of Commerce 15-17).

IDEA Encryption

The International Data Encryption Algorithm (IDEA) was developed by Xuejia Lai and James Massey in an attempt to replace DES with a stronger algorithm and "to promote research"(Lai). They succeeded in improving the security, so that IDEA is considered immune to differential, linear, and algebraic attacks. IDEA's biggest found flaw is a set of "2^{51} weak keys [...] use of such a key during encryption could be detected easily and the key recovered" (RSA Labs 3.6.7)

Mediacrypt, the current operator of licenses and technical specifications for IDEA, acknowledges these weak keys. They continue to say that the keys are defined as weak if there are

3 The NIST document SP 800-38A describes the counter as exactly what would be expected, with a slight modification. It removes *n* bits from the incrementer and sets permanently to a specific value. Then the counter is initialized to a starting value, and every block from then on the non-permanent section is incremented. This allows for $2^{\text{size of incrementer}}$ unique counters.

more than fourteen 0's (counting the connection from the start to the end). With pure random chance there exists 2^{-76} (or approximately $1 / (7 * 10^{21})$) chance of a weak key being randomly selected (Weak_keys_0206).

Walk though of IDEA Encryption

The IDEA cipher uses 64 bits of plaintext and a 128 bit key. The resulting ciphertext is also 64 bits. Fifty-two 16-bit subkeys are derived from the original key and are used throughout the 9 rounds of encryption. These subkeys gives a larger sense of randomness to the ciphertext, thus making it harder to derive information from the ciphertext itself (the issue of padding remains). Each block of 64 bits is then ran through the same process using the same subkeys (Menezes 263-264). This implementation utilizes the ECB version of padding. The following implementation is taken from the Handbook of Applied Cryptography page 264.

The creation of the encryption subkeys is a relatively trivial process. Divide the 128 bits into 8 sets of 16 bits each. Each set of 16 bits becomes the next subkey. Variety is gained by when that set of 128 bits has been used, rotate the 128 bits left 25 times. From there group the keys into groups of 8, marking the first group for round 1, ..., last group (will be a set of 4 subkeys) for round 9. Next, divide the 64 bits of entry text into 4 16-bit sections.

The creation of the decryption subkeys is a series of taking inverses of the encryption subkeys and modifying based on how they get used in the main part of IDEA. The order of pulling the encryption subkeys is such that the last key used to encrypt the data is the first key

used to decrypt the data. Pulling the encryption keys in this order in groups of six, each key goes through the following transformation:

- If it is the first or fourth key, take the multiplicative inverse
- If it is the second or third key, take the additive inverse
- finally the fifth and sixth keys remain the same

After the initial work is complete, the main part of IDEA can begin. In the following description it is critical to note that: annotations use of +=, *=, and XOR= are all C++/Java-like annotations. Also each key is of the current round (i.e., first key = first key of current round), all multiplications are done MOD $2^{16}+1$ (with 0 being 2^{16}) and all additions are done MOD 2^{16}. Finally since this can be used for both encryption and decryption, entry text is the input, and

For 8 rounds:

1. entry text piece 1 *= first key
2. entry text piece 4 *= fourth key
3. entry text piece 2 += second key
4. entry text piece 3 += third key
5. Creating 3 temporary variables

   temp1 = fifth key * (entry text piece 1 XOR entry text piece 3)

   temp2 = sixth key * (temp1 + (entry text piece 2 XOR entry text piece 4))

   temp3 = temp1 + temp2
6. entry text piece 1 XOR= temp2

7. entry text piece 4 XOR= temp3
8. TEMP piece = entry text piece 2 XOR temp3
9. entry text piece 2 = entry text piece 3 XOR temp2
10. entry text piece 3 = TEMP piece

For the last round (9):

1. Output text piece 1 = entry text piece 1* first key
2. Output text piece 4 = entry text piece 4 * fourth key
3. Output text piece 2 = entry text piece 3 + second key
4. Output text piece 3 = entry text piece 2 + third key

Personal Difficulties

IDEA was the first algorithm that was coded for the code section of my project. Throughout the coding of IDEA, various bumps occurred along the way. The first was the necessity of creating the BitHolder class. This class was made to allow the cycling of bits and to allow the bit-size to be variable, for further information see the code documentation. The use of the keyword "unsigned" became an obvious advantage to me, as two's complement complicated some of the computations particularly the XORs. Also during this first exploration of encryption, the inherent difficulty of creating what appears to be garbage proved debugging to be a very difficult task. Throughout this project I was unable to find a solution to this problem of how to debug something without knowing what it's results would be. As an exception to this, The

Handbook of Applied Cryptography was an extremely useful resource as it gave samples. This, however, at times became a double edged sword. There were moments through the coding, that since I knew what the result should have been, I was somewhat blinded by what may be causing an error. This resulted (at the beginning) in some quick patches, that proved disastrous.

During this time of creating my IDEAENCRYPTOR, my advisor also requested that I load files into memory. Files were to be loaded into memory so that IO caching would not interfere with the timing element. In an effort to make a reasonable encryptor (one that could do any file size), I began to make the option to encrypt files with or without using the hard disk. This both allows users to encrypt as large as files as they want (as the file is only loaded into memory as needed) and allowed me to test for speed more accurately.

Asymmetric Cipher Overview

Asymmetric ciphers are unique ciphers because they have separate keys for encryption and decryption. The two keys very often get named public and private key (sometimes the ciphers can be referred to as public/private key ciphers). Each key holds a separate role in the encryption process.

The public key's job is to allow others to encrypt a message. Interestingly these keys "need not be kept secret, and, in fact, may be widely available" (Menezes 283). The problem this introduces is now authenticating who originated the message. Without means of authentication man-in-the-middle attacks become exceedingly easy. This authentication is not covered in the scope of my project and is only mentioned here for informative purposes.

The private key's job is to allow the decryption of a message assuming it was encrypted using the related public key. For security to work in this case, "the task of computing [the private key] from [the ciphered message must be] computationally infeasible (Menezes 283). Otherwise all messages to the owner of the key pair can be intercepted, there by violating all that encryption is trying to do.

Typically asymmetric encryption is used to convey small pieces of data such as "keys [...,] credit card numbers, and PINS" (Menezes 283). Asymmetric ciphers are primarily used for these smaller pieces of data because of the asymmetric encryption process is slow, for further information see the chart in the appendix outlining the relative speed of the three ciphers.

Asymmetric keys can also be used in situations where many individual secure communications are required. For example if a network is used so that every person (n) on the network needs to communicate with every other person in a secure fashion how many keys would be needed? In a symmetric key setup it would require $n(n-1)/2$ keys. The same number of people would require only n pairs of keys if done asymmetrically (Assche 32). Since each asymmetric key requires 2 keys the final comparison of asymmetric vs symmetric is $n < n(n-1)/4$ for all n greater than 5. Given most modern networks have significantly more than 5 people on the network, there is much to be saved through asymmetric keys.

RSA Encryption

RSA encryption "is the most widely used public-key cryptosystem"(Menezes 285). The name RSA comes from the last initials of the inventors: R. Rivest, A. Shamir, and L. Adleman (Menezes 285). Dr. Ronald Rivest commented in an email that "the motivation for the RSA work was the Diffie-Hellman 'New Directions in Cryptography' paper". When asked about strengths and weaknesses in actually implementing RSA, Dr Rivest commented that "RSA is [...] well-understood, and factoring seems to be reasonably well-understood. But RSA keys are a bit larger than one would like in many applications". This implies the speed penalty is known, but as mathematics progress RSA still holds strong.

RSA is thought to be a "computationally-secure cipher, as it relies on the difficulty to factor large number" (Assche 30). This means that the security of RSA relies on taking

noticeably longer to factor a number, than attempting to multiply the two. Dr. Assche suggests on page 30 of Quantum Cryptography and Secret-Key distillation "[t]here is currently no proof that RSA and factoring are computationally equivalent, although many believe so". Opposite of Dr. Assche's view on RSA security, RSA Laboratories has much more confidence in RSA. In RSA laboratories' article entitled "A Cost-Based Security Analysis of Symmetric and Asymmetric Key Lengths", the time and space needed to break RSA encryption is analyzed. They calculated that it would take 8000 MIPS-Years[4] and 2.3 GBytes of memory to break RSA using a 512-bit key (using modern factorization techniques). The time and memory requirements raises greatly as the bits increase as presented in the following table, derived from data found from RSA laboratories.

Bits of encryption level	Time (in MIPS-Years)	Memory (in GBytes)
576	8.72E+004	7.6
640	8.08E+005	23.0
704	6.68E+006	66.7
768	4.80E+007	177.1
1024	5.60E+010	6,095.0
2048	7.20E+019	207,000,000.0

Walk through of RSA encryption

The first step in running RSA, is to develop the public and private key for a given user. The The Handbook of Applied Cryptography by Menezes, provides the following outline of how to do so:

4MIPS-Year: Millions of Instructions Per Second running for one Year

- Find two large primes each approximately the same size (in bits or decimal places)
- The modulus to be used is product of these two primes
- Calculate a temporary number equal to the product of each prime minus 1
- Also choose a random number so that 3 <= this number <= the modulus, and such that the product of the primes and this number do not share any factors. (Assche 29)
- Use Euclidean's extended algorithm to find the number so that it is:
    - positive number less than the temporary number and greater than 1
    - and: the product of the temporary number and this number = 1 mod the temporary number
- The two part public key is then the product in modulus and the chosen random number
- The private key is the result of the extended Euclidean algorithm

After the keys are established the encryption / decryption process is as follows:

- To Encrypt
    - Take the message represented as a number and raise to the power of the random number
    - Then mod this result by the modulus
- To Decrypt
    - Take the ciphered message as a number raised to the private key
    - Then mod this result by the modulus

The difficulty (and security) comes from the work to calculate primes of sufficient size.

This adds an extremely noticeable amount of time to the execution of this code (see the appendix). The overall security of this algorithm rests on two principles: the understanding of how prime numbers work and pseudorandom number generators that work. If there were discovered a maximum prime, or factorization of large numbers into primes were simplified RSA's security would crumble. As the most common problem with pseudorandom number generators, if the random numbers can be easily predicted then it would be possible to recreate the chosen numbers and therefore circumventing the security.

Personal Difficulties

This is the only section of my project that will not correctly decrypt in a given situation. It will compute the correct value every time but, has potential to lose some bytes. This occurs when the original text contains a set of leading zeros. It is important to note here that leading zeros will be defined by the value of the modulus, therefore not calculable before hand.

I attempted various ways of passing how many sets of integers would be in each block. Recovering that information became difficult (if that information is encrypted then how-to recover). There are some checks inside of my code that seem to catch some of them but does not catch all of them. As a direct result of this project being primarily research, I decided to leave the bug in the code.

Stream Cipher Overview

Stream ciphers differ from the other ciphers because they will encrypt one byte at a time. They allow for any size message to be encrypted without the need for padding or any other method of message resizing. Therefore the output size will always be the same size as the original message. Usually these ciphers use the XOR operator, comparing the message to a preexisting table. Making stream ciphers generally faster than other ciphers, assuming some of the work is done in advance (however, see the appendix as mine was slower than the block cipher). (Menezes 191)

This specialization allows for stream ciphers to be used in places where other ciphers would not work as well. Stream ciphers are known to "have limited or no error propagation[5] [... they] may also be advantageous is situations where transmission errors are highly probable" (Menezes 191). Error propagation can be ignored since every bit is handled independently of every other bit. Error propagation can be an issue in other ciphers, as the created encrypted text can be used to encrypt later data, see the block cipher section on padding and cipher modes for some examples. This means that stream ciphers have the capability to be used for secure communications, such as cell phones and streamed internet data. Stream ciphers tend to be propriety in nature, and specific to use and company. Unfortunately, due to these

5 Error propagation is where one mistake cascades to cause further mistakes. In cryptography it refers specifically if something goes wrong then everything after it goes wrong. Stream ciphers do not worry about this since they deal with one byte at a time, and byte A has no bearing on byte B.

specific uses "there are relatively few fully-specified stream ciphers in open literature" (Menezes 191).

SCREAM Cipher

The SCREAM cipher was developed by Shai Halevi, Don Coppersmith, and Charanjit Jutla. It was designed to be an enhancement of SEAL cipher designed by Coppersmith and Rogaway. SEAL, even in its most advanced code, was found to be possibly cracked after approximately 2^{44} bytes of output; however, it was designed to have 2^{48} safe output bytes. SCREAM is thought to be safe up to about 2^{80} bytes, allowing a much larger usability. (Halevi Section 1.1)

Walk through of SCREAM Cipher

The SCREAM cipher, like other stream ciphers, sets up a table that can be used to XOR the plaintext with. The SCREAM cipher takes a seed to establish a key, then uses the key to create more security- a nonce[6], which is then used to create the tables. Below is summarized the steps found in "Scream: a software-efficient stream cipher " written by Shai Halevi, Don Coppersmith, and Charanjit Jutla . The Rijndael S-box and the Rijndael S-box2 are both calculate substitution charts available online or elsewhere and therefore not included here (although code for it can be found in the code section of the appendix). The Rijndael S-box2 XOR the look up with 21 first (for mathematical reasons beyond me). Prior to describing the algorithm, there are

6The Alliance for Telecommunications Industry Solutions defines a nonce as: In cryptography, a time-variant parameter, such as a counter or a time stamp, that is used in key management protocols to prevent message replay and other types of attacks. [After X9.42] (www.atis.org)

two functions which must be defined.

Function Gsm

- Replace each element in the matrix by either the Rijndael S-box or Rijndael S-box2 depending in the subscript of s)
- Next rotate the second row of the matrix one byte to the right
- Next replace column by matrix m (depending on the subscript of m)

Function F taking a 16 byte input

- Make 2 matrices from x, each a 2 X 4 called A and B
- Set B to be B XOR Gs_2m_2 A
- Set A to be Gs_1m_1 A
- Rotate B by two columns
- Set A to be A XOR Gs_2m_2 B
- Set B to be Gs_1m_1 B
- Finally return the modified 16 bytes

Key setup taking a 16 byte seed making a 16 X 16 byte table

- First set *a* to be the seed
- Set *b* to be function F of (*a* XOR the first 16 bytes of *pi*)
- Do the following for 16 iterations
  - Set *a* to *a* through function F 4 times XOR *b*
  - Then set the next available row of the table to *a*

Nonce creation from the table created in the key setup taking a 16 byte input

- *z* is set to the result of the F function of *input* XOR the first row of the table, and then the F function on that result
- *y* is set to the result of the F function of *z* XOR the third row of the table, and then the F function on that result
- *a* is set to the result of the F function of *y* XOR the fifth row of the table, and then the F

function on that result

- $b = x$ is set to the result of the F function of a XOR the seventh row of the table
- Do the following for 8 iterations (0 -7)
  - set b to the F function of b XOR row (2 * iteration) of the table
  - row (2 * iteration) of the table is set to row (2 * iteration) of the table XOR a
  - row (2 * iteration + 1) of the table is set to row (2 * iteration + 1) of the table XOR b
- Save the values x,y,z and the new table created for the main loop

The main loop of the SCREAM cipher taking the x,y,z and the table from nonce creation

- Set *index* to be 0
- Do the following as long as you need more bytes
  - Do the following for 16 iterations (0 -15)
    - x = function F of (x XOR y) XOR z
    - output x XOR table row (*index* MOD 16)
    - if *iteration* = 0 or *iteration* MOD 4 = 2
      - rotate y by 8 bytes
    - else if *iteration* MOD 4 = 1
      - rotate each half of y by 4 bytes
    - else if *iteration* not 15
      - rotate each half of y right by 3 bytes
- y is set to function F of y XOR z
- z is set to function F of y XOR z
- table row *index* is set to function F of that table row *index*
- *index* is set to *index* + 1 MOD 16

Personal Difficulties

SCREAM overall, was not a difficult cipher. Since the creators made the cipher with

software in mind, pseudocode was also given with the documentation. It was the case that some of the pseudocode was difficult to read as they placed advanced math references inside of the pseudocode. These references were to assist in proving the security level, while to me (not as a mathematician, but as a computer scientist) they confused me. After I was able to isolate these proving references from the software pseudocode implementing simply came with the normal debugging processes.

```
#ifndef _ANENCRYPTOR_H
#define	_ANENCRYPTOR_H

#include <iostream>
#include "READERS.h"
#include "WRITERS.h"
using namespace std;

class anEncryptor
{
public:
   //used to set input/ ouput to be disk based or memory based
   anEncryptor(const bool &inputUseHd, const bool &outputUseHd);
   virtual ~anEncryptor();//to destroy input / output private variables

   //up to the algorithm to handle
   virtual void decrypt() = 0;
   virtual void encrypt(string filename) = 0;

protected:
   READERS *input;
   WRITERS *output;
};//end class anEncryptor

#endif /* _ANENCRYPTOR_H */
```

```
#include "anEncryptor.h"
#include "diskReader.h"
#include "memReader.h"
#include "diskWriter.h"
#include "memWriter.h"

anEncryptor::anEncryptor(const bool &inHd, const bool &outHd)
{
  /*
   *inHd determines diskReader or memReader
   *outHd determines diskWriter ir memWriter
   */

  if(inHd)
    {input = new diskReader();}
  else
    {input = new memReader();}

  if(outHd)
    {output = new diskWriter();}
  else
    {output = new memWriter();}
}//end anEncryptor(bool, bool)

anEncryptor::~anEncryptor()
{
  /*
   *delete the pointers
   */
  delete input;
  delete output;
}
```

```
#ifndef _BITHOLDER_H
#define	_BITHOLDER_H

#include <vector>
#include <iostream>
using namespace std;

class BitHolder
{
public:
   BitHolder();//constructor calls <vector>'s constructor to hold 0 at first

   void clear();//clears all bits, and sets used to 0

   // cycle all bits left/ right, amount times
   bool cycleLeft(const unsigned int &amount);
   bool cycleRight(const unsigned int &amount);

   unsigned int getUsed() {return myBits.size();}
   /********************** Template definitions ********************/
   template<typename T>
        bool fill(T& fillMe, const int &startAt)
   {
      /*
       *fillMe is a holder for the bits to be loaded
       *startAt is the index of myBits (a vector) to begin loading at
       *
       *return false if would leave bounds of loaded variables, true otherwise
       */

      if((sizeof(fillMe)*8) + startAt > myBits.size() + 1)
      {
         cout <<"Cannot access bits that have not been set:  returning false in BitHolder::fill";
         cout <<endl<<"Results have become invalid."<<endl;
         return false;
      }//end if ((sizeof(fillMe)*8) + startAt > used)

      fillMe =0;
```

```
      for(unsigned int x =0; x < sizeof(fillMe)*8 - 1; ++x)
      {
        if(myBits[x+startAt])
          {++fillMe;}

        fillMe <<= 1;
      }//end for(int x =0; x < sizeof(fillMe)*8 -1; ++x)

      if(myBits[startAt - 1 +sizeof(fillMe)*8])
        {++fillMe;}//get the lowest bit, seperate because no shifting needed

      return true;
}//end fill<typename T>(T& fillme, int startAt)

template <typename T>
  bool load(const T &toLoad)
    {return load(toLoad, sizeof(toLoad));}

template <typename T>
bool load(const T &toLoad, unsigned int const &numOfBytes)
{/*
 *loads the toLoad variable into the next availabe spots of myBits
 *numOfBytes is how many bytes from toLoad that will be inserted.
 *
 *returns false if numOfbytes > sizeof(toLoad)
*/

  if(numOfBytes > sizeof(toLoad))
  {
    cout << "toLoad does not have " << numOfBytes << "in it.";
    cout << endl << "bool BitHolder::load(T toload, int numOfBytes)";
    return false;
  }//end if(numOfBytes > sizeof(toLoad))

  int sizeInBits = numOfBytes *8;
  T bitPlace =1;

  /*
```

```
    * Used to since push back would otherwise push back words
    * i.e. 123 would get pushed 321 since want the highest bit
    *use BitPlace to store the highest power of two
    */
   for(unsigned int x =0; x < sizeof(toLoad)*8 -1; ++x)
      {bitPlace <<= 1;}//end for(int x =0; x < sizeof(toLoad)*8 -1; ++x)

   for(int x =0; x <sizeInBits; ++x)
   {
     if(bitPlace == (bitPlace & (toLoad<<x)))//checks if highest bit = 1
     {myBits.push_back(true);}
     else
     {myBits.push_back(false);}

     //toLoad <<= 1;
   }//end for(int x =0; x <sizeInBits; ++x)
   return true;
}//end bool load(T toLoad, int const &numOfBytes)
   /*********************end Template definitions *********************/


private:
   vector<bool> myBits;//To allow any number of bits that memory supports

   //Cycles left/right one time.  Helper functions to the publics
   bool cycleLeft();
};//end class BitHolder

#endif /* _BITHOLDER_H */

#include "BitHolder.h"

BitHolder::BitHolder()
{
   myBits.resize(0);//clears the bits
}//end BitHolder()

void BitHolder::clear()
{//erase all bits, and set used = 0
```

```
  myBits.erase(myBits.begin(), myBits.end());
  myBits.clear();
}//end BitHolder::clear()

bool BitHolder::cycleLeft(const unsigned int &amount)
{/*
 *The public use of cycle allows for the amount to dictate how many individual
 *cyclelical movements there are
 */
  for(unsigned int x =0; x < amount; ++x)
  {
    if(!cycleLeft())
      return false;
  }//end for(int x =0; x < amount; ++x)

  return true;
}//end bool BitHolder::cycleLeft(unsigned int amount)

bool BitHolder::cycleLeft()
{
  /*
   *Cycle left is relativly easy.  Take the first one, and move to the end.
   *Used a difference of [0] annotation and begin() annotation, because of
   *vector's function definitions, could not make match.
   */
  if(myBits.empty())
  {return false;}

  myBits.push_back(myBits[0]);
  myBits.erase(myBits.begin());

  return true;
}//end bool BitHolder::cycleLeft()
```

```
#ifndef _DISKREADER_H
#define	_DISKREADER_H

#include "READERS.h"

class diskReader : public READERS
{
public:
   /*
    *Given there is no difference between a disk reader and an ifstream
    *these functions simply call ifstreams versions.
    *Used so that encryptors can have pointers to memReaders or diskReaders
    *
    *Further information by looking up ifstream.
    */
   void close();//closes the file
   void open(const string &t);//opens the fle
   void read(char* into, const int &num);//reads from the file to into
};

#endif /* _DISKREADER_H */
```

```
#include "diskReader.h"
/*
 *The diskReader, effectivly acts as a wrapper to ifstream. Short and sweet.
 *for more infomation see the ifstream documentation
 */
void diskReader::close()
{
   ifstream::close();
   length = 0;
}

void diskReader::open(const string &t)
{
   ifstream::open(t.c_str());
   filename = t;
}

void diskReader::read(char *into, const int &num)
{ifstream::read(into, num);}
```

```
#ifndef _DISKWRITER_H
#define	_DISKWRITER_H
#include "WRITERS.h"

class diskWriter : public WRITERS
{
public:
   /*
    *Given there is no difference between a disk writer and an ofstream
    *these functions simply call ofstream's versions.
    *Used so that encryptors can have pointers to memWriters or diskWriters
    *
    *Further information by looking up ofstream.
    */
   void close();//close the file
   void open(const string &t);//opens the file
   void write(const char *from, const int &num);// writes to the file
};

#endif /* _DISKWRITER_H */
```

```
#include "diskWriter.h"
/*
 *The diskReader, effectivly acts as a wrapper to ifstream. Short and sweet.
 *For more details view ofstream's documentation.
 */
void diskWriter::close()
{ofstream::close();}

void diskWriter::open(const string &t)
{ofstream::open(t.c_str());}

void diskWriter::write(const char *from, const int &num)
{ofstream::write(from, num);}
```

```
#ifndef _IDEAENCRYPTOR_H
#define        _IDEAENCRYPTOR_H

#include "BitHolder.h"
#include "anEncryptor.h"

class IDEAencryptor : public anEncryptor
{
public:
   IDEAencryptor(const bool &inUseHd, const bool &outUseHd);//calls anEncryptor to use
memory or IO
   virtual ~IDEAencryptor();//deletes the pointers

   //to encrypt and decrypt
   void decrypt();
   void encrypt(string filename);

private:
   BitHolder *myKey; //The original key as typed in

   /*
    *The calculation of various subkeys need to do a lot of work.  Note that
    *calculateSubkeys makes the encryption subkeys, but these are used in the
    *decrypt subkeys as well.  The calculation of the decrypt subkeys requires
    *substantial work.
    */
   void calculateDecryptSubkeys(unsigned short result[]);
   void calculateDecryptSubkeysRound1and9(const unsigned short encryptionSubKey[],
        unsigned short result[], const int &round);
   void calculateDecryptSubkeysRound2to8(const unsigned short encryptionSubKey[],
        unsigned short result[]);
   void calculateSubkeys(unsigned short result[]);

   void cryptHelper(unsigned short subkeys[]);

   unsigned short euclideanAlgorithm(unsigned int b);

   /*
```

```
    *The IDEA encryption process is the same for encrypt and decrypt, therefore the
    *name of "mainIDEA".  The changes are a result of the inputs, encrypt or decrypt
    *subkeys, and which file.
    */
   void mainIDEA(unsigned short result[], const unsigned short keys[]);
   void mainIDEAround1to8(unsigned short result[], const unsigned short keys[]);
   void mainIDEAround9(unsigned short result[], const unsigned short keys[]);

   //The mod functions are commonly used through the IDEA encryption.
   short modMult(const unsigned short &a, const unsigned short &b);
   short modAdd(const unsigned short &a, const unsigned short &b);

   void setKey();//sets the Key and ensures it's length is proper.
};//end class IDEAencryptor

#endif /* _IDEAENCRYPTOR_H */
```

```
#include "IDEAencryptor.h"
#include <iostream>
#include <algorithm>
using namespace std;

IDEAencryptor::IDEAencryptor(const bool &inUseHd, const bool &outUseHd)
: anEncryptor(inUseHd, outUseHd)
{
   /*
    *Create a bit holder, and artificially load the key
    */
   myKey = new BitHolder();
   /*
    *This code is to make ease of testing and use, and is therefore removeable
    *for any "real" application
    */

   for(unsigned short x =1; x <= 8;x++ )
      myKey->load(x);
}//end Constructor

IDEAencryptor::~IDEAencryptor()
   {delete myKey;}

void IDEAencryptor::calculateDecryptSubkeys(unsigned short result[])
{/*
 *There are 52 subkeys at the end, and notice that rounds 1 and 9
 *are similiar except last 2 keys.  Rounds 2-8 are all identical
 */
   unsigned short encryptionSubkey[52];
   calculateSubkeys(encryptionSubkey);

   calculateDecryptSubkeysRound1and9(encryptionSubkey, result, 1);
   calculateDecryptSubkeysRound2to8(encryptionSubkey, result);
   calculateDecryptSubkeysRound1and9(encryptionSubkey, result, 9);
}//end void IDEAencryptor::calculateDecreyptSubkeys(BitHolder result[])

void IDEAencryptor::calculateDecryptSubkeysRound1and9(const unsigned short
```

```
encryptionSubkey[],
    unsigned short result[], const int &round)
{
    const int subkey = (9-round)*6;//the encryption subkey that we start from
    const int load = (round - 1)*6;//this is used as to where start result from

    for(int x =0; x < 4; ++x)
    {//round 9 do not load the last 2 so only 4 for now
      switch(x)
      {
        case 0:
        case 3:
          result[load + x] =
              euclideanAlgorithm(encryptionSubkey[subkey + x]);
          break;

        case 1:
        case 2:
          result[load + x] = ~encryptionSubkey[subkey + x];
          ++result[load + x];
          break;
      }//end switch(x)
    }//end for(int x =0)

    if(round ==1)
    {
      result[4] = encryptionSubkey[subkey -2];
      result[5] = encryptionSubkey[subkey -1];
    }
}//end void calculateDecrytSubkeysRound1and9

void IDEAencryptor::calculateDecryptSubkeysRound2to8(const unsigned short
encryptionSubkey[],
    unsigned short result[])
{
  /*
   *Rounds 2 and 3 both need the additive inverse. Used 2s complement for speed.
   *Other option (more readable) would have been to multiply by -1
   */
```

```
    unsigned int subkey, place;

    for(int round = 2; round < 9; ++round)
    {
      subkey = (9-round)*6;
      place = (round -1)*6;

      /////////////// 1 /////////////////
      result[place] = euclideanAlgorithm(encryptionSubkey[subkey]);
      ////////////////////////////////////

      ///////////////// 2 /////////////////
      result[place + 1] = ~encryptionSubkey[subkey+2];
      ++result[place + 1];
      ////////////////////////////////////

      ///////////////// 3 /////////////////
      result[place + 2] = ~encryptionSubkey[subkey+1];
      ++result[place + 2];
      ////////////////////////////////////

      /////////////// 4 /////////////////
      result[place + 3] = euclideanAlgorithm(encryptionSubkey[subkey+3]);
      ////////////////////////////////////

      ///////////////5 & 6/////////////////
      result[place + 4] = encryptionSubkey[subkey -2];
      result[place + 5] = encryptionSubkey[subkey -1];
      ////////////////////////////////////
    }//end for(int round = 2; round < 9; ++round)
}//end void IDEAencryptor::calculateDecryptSubkeysRound2to8(BitHolder encryptionSubkey[],
BitHolder result[])

void IDEAencryptor::calculateSubkeys(unsigned short result[])
{
    /*
    *Not needed since running timing, and do not want to type in values
    *Kept in case someone else would want to use this program.
     *
```

```
    *Note Also this assumes a char is one byte for all '16s' below
    */
    while(myKey->getUsed() != 128)
    {
      cout << "Key must be 128 bits long (16 chars)\n";
      setKey();
    }//end while (key.getUsed() != 128)

    for(unsigned int round = 0; round < 52; ++round)
    {
      //every 8th round after the first rotate the keys
      //so that a new set can be pulled out
      myKey->fill(result[round], (round % 8)*16);
      //every eigth cycle left 25
      //the +1 so does not happen first time
      if( ((round+1) % 8 == 0))
        myKey->cycleLeft(25);
    }//end for(round < 52

    //destroy key because this is the only place where it is used
    myKey->clear();
}//end void IDEAencryptor::calculateSubkeys(BitHolder key)

void IDEAencryptor::cryptHelper(unsigned short subkeys[])
{
    /*
     *The 8's below are calculated from sizeof(short) * length of array = 8
     */
    unsigned short fromfile[4];

    input->read((char *)fromfile, 8);
    mainIDEA(fromfile, subkeys);
    output->write((char*)fromfile, 8);
}//end cryptHelper

void IDEAencryptor::decrypt()
{
    /*Notice use of crypt helper in for loop, and the last one handled as a unique case
     *
```

```
    *The decrypt algorithm is fairly easy after the encrypt part.
    *The same mainIDEA is used, with input being ciphered and a new key.
    *For our purposes, pre-naming works well.
    */
   input->open("IDEAciphered");
   output->open("IDEAdeciphered");

   unsigned short subkeys[52];
   unsigned long long const counter = (input->getLength() / 8) -1;

////////////////////////////////////////////

   calculateDecryptSubkeys(subkeys);

   /*long long used to handle BIG files
    *Notice the -1.  The final group of bytes, may or may not have been padding
    *therefore they require special attention.
    */
   for(unsigned long long x =0; x < counter; ++x)
   {
      cryptHelper(subkeys);
   }//end for(x < length)

   /*
    *Reading the last 8 is the same as before, but chopping off some of the bytes
    *Note:  fromfile[3] & oxff00>>8 looks odd but due to endianess need the first
    *part of the final short.
    */
   unsigned short fromfile[4];
   input->read((char *)fromfile, 8);
   mainIDEA(fromfile, subkeys);
   output->write((char *)fromfile, 8 - ((fromfile[3] & 0xff00)>>8));

   input->close();
   output->close();

}//end void IDEAencryptor::decrypt()

void IDEAencryptor::encrypt(string filename)
```

```
{
  /*
   *Calculate the encryption subkeys and the shove through main IDEA
   *
   *Notice use of crypt helper in for loop, and the last one handled as a unique case
   */
  input->open(filename);
  output->open("IDEAciphered");

  unsigned short subkey[52];
  unsigned long long const counter = input->getLength() / 8;
  unsigned int const leftOver = (input->getLength() % 8);

  calculateSubkeys(subkey);

  for(unsigned int x =0; x < counter; ++x)
  {//for every 8 do calc and file load
     cryptHelper(subkey);
  }//end  for(int x =0; x < filesize / 8; ++x)
  ////////////////////////////////////////
  /*
   *last insert special:
   *if leftOver = 0, insert 8 chars of 8
   */

  unsigned char temp[8] = {8,8,8,8,8,8,8,8};

  if(leftOver !=0)
  {
     input->read((char *)temp, leftOver);
     for(int x = leftOver; x < 8; ++x)
     {temp[x] = (8 - leftOver);}
  }//end if(leftOver != 0)

  mainIDEA((unsigned short *)temp, subkey);
  output->write((char *)temp, 8);

  output->close();
  input->close();
```

```
}//end void IDEAencryptor::encrypt()

void IDEAencryptor::mainIDEA(unsigned short result[], const unsigned short keys[])
{
  /*
   *result's length is always 4
   *all * and + are mod 2^16
   *
   *Calculated as such
   *for(round 1 to 8)
   *  for n 1 to 4:
   *     if n is 1,4:   result(n) = result(n) * key(round)(n)
   *     else           result(n) = result(n) + key(round)(n)
   *  loop
   *
   *  temp0 = key(round)(5) * (result(1) XOR result(3))
   *  temp1 = key(round)(6) * (temp0 + (result(2) XOR result(4)))
   *  temp2 = temp0 XOR temp1
   *
   *  result(1) = result(1) XOR temp1
   *  result(4) = result(4) XOR temp2
   *        a = result(2) XOR temp2
   *  result(2) = result(3) XOR temp1
   *  result(3) = a
   *loop
   *
   *result(1) = result(1) * key(9)(1)
   *result(4) = result(4) * key(9)(4)
   *result(2) = result(3) + key(9)(2)
   *result(3) = result(2) + key(9)(3)
   */
  mainIDEAround1to8(result, keys);
  mainIDEAround9(result, keys);
}//end void IDEAencryptor::mainIDEA(BitHolder result[], const BitHolder keys[])

void IDEAencryptor::mainIDEAround1to8(unsigned short result[], const unsigned short keys[])
{
  unsigned short t[3];
  unsigned short temp;
```

```
  for(short round = 0; round < 8; ++round)
  {
    ////////////part a/////////////////
    result[0] = modMult(result[0], keys[round*6]);
    result[3] = modMult(result[3], keys[round*6 + 3]);
    result[1] = modAdd(result[1], keys[round*6 + 1]);
    result[2] = modAdd(result[2], keys[round*6 + 2]);
    ////////end part a/////////////////

    ////////////part b/////////////////////
    temp = result[0] ^ result[2];
    t[0] = modMult(temp, keys[round*6 + 4]);

    temp = result[1] ^ result[3];
    temp = modAdd(temp, t[0]);
    t[1] = modMult(temp, keys[round*6 + 5]);

    t[2] = modAdd(t[1], t[0]);
    ////////// end part b/////////////////////

    ////////////////// part c ////////////
    result[0] ^= t[1];
    result[3] ^= t[2];
    temp = result[1] ^ t[2];
    result[1] = result[2] ^ t[1];
    result[2] = temp;
    ////////////////end part c////////////
  }//end for(short round = 0; round < 8; ++round)
}//end void IDEAencryptor::mainIDEAround1to8(unsigned short result[], BitHolder keys[])

void IDEAencryptor::mainIDEAround9(unsigned short result[], const unsigned short keys[])
{
  unsigned short const temp = result[1];

  result[0] = modMult(result[0], keys[48]);
  result[3] = modMult(result[3], keys[48+ 3]);
  result[1] = modAdd(result[2], keys[48+ 1]);
  result[2] = modAdd(temp, keys[48+ 2]);
```

```
}//end void IDEAencryptor::mainIDEAround9(unsigned short result[], BitHolder keys[])

short IDEAencryptor::modAdd(const unsigned short &a, const unsigned short &b)
{
  return (a + b) & 0xFFFF;// this is the same (but faster) of c % 0x10000
}//end short IDEAencryptor::modAdd(const unsigned short &a, const unsigned short &b)

short IDEAencryptor::modMult(const unsigned short &a, const unsigned short &b)
{
  /*
   *  Handbook of applied cryptography page 264 has the psudeo code for
   * this modified modular multipicliation as it applies to IDEA cipher.
   * Note 7.104
   */
  int result;//made int to handle temporarily larger numbers

  if(a == 0)
  {result = 0x10001 - b;}
  else if(b==0)
  {result = 0x10001 - a;}
  else
  {
    unsigned int const c = a * b;
    result = (c & 0xFFFF) - (c >> 16);

    if(result < 0)
      result += 0x10001;
  }//end else

  return (result & 0xFFFF);//removes the non unsigned short part
}//end short IDEAencryptor::modMult(short a, short b)

unsigned short IDEAencryptor::euclideanAlgorithm(unsigned int b)
{
  /*
   *The larger spots allow for the algorithm below to handle the multiplication
   *appropiatly.
   *
   *The euclidean algorithm below is modeled after the algorithm on page 67
```

```
*of the Handbook of Applied Cryptography.
*
*Since here always sending back y2 and a always = 2^16 +1
*
*modeled as such:
*
*INPUT two non negative a,b: a>= b
*if b =0: return 0
*
*start with x2 = y2 = 0: x1 = y1 = 1
*while b > 0
*  q = floor(a/b)
*  r = a - qb
*  x = x2 -qx1
*  y = y2 - qy1
*
*  a=b, b=r, x2=x1, x1=x, y2=y1, y1=y
*loop
*
*return y2
*/
unsigned int a = 0x10001;
int x,y,x1,x2,y1,y2,q,r;

if(b==0)
{return 0;}

x2=y1 =1;
x1 = y2 = 0;

while(b > 0)
{
   q = a/b; // only want truncated version so happy here
   r = a - (q*b);
   x = x2 - (q*x1);
   y = y2 - (q*y1);

   a = b;
   b = r;
```

```
      x2 = x1;
      x1 = x;
      y2 = y1;
      y1 = y;
    }//end while(b>0)

    if((short)y2 < 0)
      y2 += 0x10001;

    return y2 % 0x10001;
}//end unsigned short IDEAencryptor::euclideanAlgorithm(short b)

void IDEAencryptor::setKey()
{//gets a key from the keyboard
    string keyFromKeyboard;
    unsigned int charsNeeded = 128 / (8 * sizeof(char));
    cout << "Please type in a key to use.\nNotice that the key must be ";
    cout << charsNeeded << " characters long.";

    cin >> keyFromKeyboard;
    while(keyFromKeyboard.length() != charsNeeded)
    {
      cout << "Length not accepted. Try again.";
      cin >> keyFromKeyboard;
    }//end while(keyFromKeyboard.length() != charsNeeded)

    for(unsigned int x =0; x < charsNeeded; ++x)
    {
      myKey->load(keyFromKeyboard[x]);
    }//end for( x < charsNeeded)
}//end void setKey()
```

```
#include <iostream>
#include "IDEAencryptor.h"
#include "RSAencryptor.h"
#include "SCREAMencryptor.h"
using namespace std;

void error();
void runProgram(string filename, string type, bool useIO, string todo);

int main(int varCount, char *vars[])
{
   bool useIO = (vars[3][0] == 'y' || vars[3][0]=='Y');

   switch(varCount)
   {
      case 5: runProgram(vars[1], vars[2], useIO, vars[4]);
      break;
      default: error();
   }
   return 0;
}//end main

void runProgram(string filename, string type, bool useIO, string todo)
{
      anEncryptor *ae = NULL;

      if(type == "RSA")
      {
         /*
          *To evaluate timing will need to take average keygentime
          *subtract results, and divide by 2. Gives good average that way
          *Needed since we must share a key...
          */
         ae = new RSAencryptor(useIO, useIO);
         ae->encrypt(filename);
         ae->decrypt();
         delete ae;
      }//end if RSA
```

```
    if(type == "SCREAM" || type == "IDEA")
    {
      ae = new SCREAMencryptor(useIO, useIO);
      if(todo == "encrypt")
        ae->encrypt(filename);
      else
        ae->decrypt();
      delete ae;
      /*
      ae->encrypt(filename);
      delete ae;
      ae = new SCREAMencryptor(useIO, useIO);
      ae->decrypt();
      delete ae;
       */
    }//end if SCREAM

    if(ae == NULL)
    {error();}
}//end runOneTest

void error()
{
  cout << "To run program please use the form:\n"<<
       "./sp_in_c <filename> <IDEA|SCREAM|RSA> <yes|no> <encrypt|decrypt>\n"<<
       "where filename is the file to encrypt then decrypt\n"<<
       "next is the type of encryption used\n"<<
       "next is to use diskIO (yes) or try to load into memory(no)\n"<<
       "then whether the encrypt or decrypt function should be called"<<
       endl;
}
```

```
#ifndef _MEMREADER_H
#define	_MEMREADER_H

#include "READERS.h"
#include <vector>
class memReader : public READERS
{
public:

   /*
    *memReaders act similiar to ifstream, with the exception that the IO occurs in RAM.
    *This is done utilizing a vector, set to the size the file to open.
    *The names are the same as all READERS to ensure that an encryptor does not care
    *where the input comes from, just the accessibility of the functions.
    */
   memReader();// sets up the vector

   void close();//closes the file
   void open(const string &t);//opens a file t
   void read(char* into, const int &num);// reads the next num bytes and returns into

private:
   vector<unsigned char> myArray;//a vector to hold bytes in memory
   //since the file is read, without changes an iterator works well here
   vector<unsigned char>::iterator pos;
};

#endif /* _MEMREADER_H */
```

```
#include "memReader.h"

memReader::memReader()
{
   /*
    *establish the vector and it's iterator
    */
   myArray.resize(0);
   pos = myArray.begin();
}//end memReader()

void memReader::close()
{
   /*
    * closes the file and frees up the memory used by the file's bytes
    */
   ifstream::close();
   myArray.clear();
   length = 0;
}

void memReader::open(const string &t)
{
   /*
    *Open the file
    *After wards read the entore file byte by byte and load each byte
    *into myArray
    */
   ifstream::open(t.c_str());
   filename = t;
   unsigned long long const length = getLength();
   myArray.resize(length);
   pos = myArray.begin();

   unsigned char c;

   for(unsigned long long x =0; x < length; ++x)
   {
```

```
      ifstream::read((char*)&c, 1);
      myArray[x] = c;
   }//end for(x < length)
}//end memReader::OPEN

void memReader::read(char* into, const int &num)
{
   /*
    *Use the iterator to read the next byte to the into array
    */
   for(int x =0; x < num; ++x)
   {
      into[x] = (*pos);
      ++pos;
   }//end for(x < num)
}//end void READ
```

```
#ifndef _MEMWRITER_H
#define        _MEMWRITER_H
#include "WRITERS.h"
#include <vector>

class memWriter : public WRITERS
{
public:
   /*
    *memReaders act similiar to ifstream, with the exception that the IO occurs in RAM.
    *This is done utilizing a vector, set to the size the file to open.
    *The names are the same as all READERS to ensure that an encryptor does not care
    *where the input comes from, just the accessibility of the functions.
    */
   memWriter();// sets up myArray

   void close();//saves and closes the current file
   void open(const string &fileName);//tries to open a file fileName
   void write(const char *from, const int &num);//writes num bytes of from
   //setSize is used to estimate the number needed.  Effectivly used to set the size
   //of the vector to the input size, yet flexible to allow for padding.
   void setSize(const unsigned long long &mysize);//establishes myArray to mysize

private:
   vector<unsigned char> myArray;// holds the bytes of the file
   //WRITERS may change over time, so it is easier to know where we were
   //writing positionally than through iterators
   unsigned long long at;//what location within the Vector for [] use
};

#endif /* _MEMWRITER_H */
```

```
#include "memWriter.h"

memWriter::memWriter()
{
   /*
    *Starts with a blank vector: myArray
    *Explicit setting
    */
   myArray.resize(0);
   at =0;
}

void memWriter::close()
{
/*
 *for the entire vector, write each byte to the harddisk
 *Probably a better way that streaming byte by byte but close is never in the clock
 *cycle count
 *
 *afterwards close file
 */
   unsigned char temp;
   for(unsigned long long x =0; x < myArray.size(); ++x)
   {
      temp = myArray[x];
      ofstream::write((char*)&temp, sizeof(temp));
   }//end for(x < size)

   ofstream::close();
}//end memWriter::CLOSE

void memWriter::open(const string &t)
{
   /*
    *Open the new file and set at to 0
    */
   ofstream::open(t.c_str());
   at =0;
```

```
}//end memWriter::OPEN

void memWriter::setSize(const unsigned long long &mySize)
{
   myArray.resize(mySize);
}//end setSize(long long mysize

void memWriter::write(const char *from, const int &num)
{
   if(at + num > myArray.size())
   {
      myArray.resize(at+num);
   }//end if(at + num > size

   for(int x =0; x < num; ++x)
   {myArray[at + x] = from[x];}

   at+= num;
}//end void memWriter::WRITE(const char*,const int)
```

```
#ifndef _READERS_H
#define	_READERS_H

#include <fstream>
#include <string>
#include <sys/stat.h>
#include <gmpxx.h>
using namespace std;

class READERS : protected ifstream
{
public:
   /*
    *all READERS at some point need to access files, so inhereting from ifstream
    *works well here (Thanks Gary!). The function names are capitilized versions of
    *ifstream to allow memReaders and diskReaders to behave in their specifc mannerisms.
    *Therefore the virtualness of the below functions.
    */
   READERS();
   virtual void close() =0;//closes file
   virtual bool is_open(){return ifstream::is_open();}
   virtual void open(const string &t) = 0;//opens file
   virtual void read(char* into, const int &num) =0;//reads num bytes to into

   // num here is the number of limbs as defined in the mpz documentation
   virtual void read(mpz_class &into, const int &num);//reads the mpz_class using read

   //regardless of the implementation, all READERS' sizes are based off of the file
   unsigned long long getLength();

protected:
   //set at every OPEN used in the getLength method to return the size.
   string filename;
   unsigned long long length;
};
#endif /* _READERS_H */
```

```
#include "READERS.h"
#include "math.h"

READERS::READERS()
{
   length = 0;//since length is unsigned
}

unsigned long long READERS::getLength()
{
   if(length ==  0)
   {
      struct stat64 fileinfo;
      stat64(filename.c_str(), &fileinfo);
      length = fileinfo.st_size;
   }
   return length;
}//end long long getLength

void READERS::read(mpz_class &into, const int &num)
{
   /*
    *The read(char*, int) SHOULD (because of virtual) call the specific implementation.
    *That is the assumption here.
    */

   into =0;
   mp_limb_t limb;
   mpz_class temp;
   long long const numOfLimbs = num / sizeof(limb);
   int const extra = (num % sizeof(limb));

   for(int x = numOfLimbs -1; x >=0; --x)
   {
      /*
       *Read backwards because of how mpz is done
       */
      read((char*)&limb, sizeof(limb));
```

```
      //effectivly bitshift over then load into into
      temp = limb;
      temp = temp * (pow(2, (8 * sizeof(limb) * x)));
      into = into + temp;
    }//end for x < num

    if(extra > 0)
    {
      read((char*)&limb, extra);
      temp = limb;
      temp = temp * (pow(2, (8 * extra * numOfLimbs)));
      into = into + temp;
    }//end if(extra > 0)
}//end read(mpz_class, const int)
```

```
#ifndef _RSAENCRYPTOR_H
#define	_RSAENCRYPTOR_H

#include <gmpxx.h>
#include "anEncryptor.h"

class RSAencryptor : public anEncryptor
{
public:
   /*
    *Constructor sets up use of hard disk vs. more memory use.
    *Also specific to RSA setups the bit level of encryption, this needs to be
    *defaulted to be compatible with using polymorphism of AnEncryptor
    *
    *The destructor must remove the temporary files
    */
   RSAencryptor(bool const inUseHD, bool const outUseHD, unsigned int level = 1024);
   ~RSAencryptor();
   /*
    *encrypt() is needed to be compatibale with AnEncryptor, however the nature of
    *public / private keys are such that two parties are needed.  Therefore encrypt
    *encrypt uses itself to call encrypt(*Bob)
    *
    *decrypt decrypts the file RSAciphered and stores it in RSAdeciphered
    */
   void encrypt(string filename);
   void encrypt(RSAencryptor* Bob, string filename);
   void decrypt();

private:
   /*
    *This assumes that noone outside of RSAencryptor should have access
    *to PUBLICKEYs
    */
   struct PUBLICKEY{mpz_class primeMult, encryptExp;};
   //This is used by encrypt to ensure the file is of suitible length
   void addPadding(string filename);
```

```
    //returns an mpz_class of decrypted data, based on the parameter data
    mpz_class decrypt(const mpz_class &data);

    //returns an mpz_class of encrypted data, based on the parameter data, and Bob's
    //public key
    mpz_class encrypt(const mpz_class &data, const RSAencryptor &Bob);

    unsigned int encryptionLevel; //how many bits worth of encryption
    void generateKeys(); //used to set up the keys

    //since timing in important, we need to save the time to generate the keys
    unsigned long long int keyGenTime;

    void makeRandPrime(mpz_class &prime);

    PUBLICKEY publicKey; // this ones publicKey
    mpz_class privateKey; //this one private key
    void removePadding(); //removes the padding on decrypt from addPadding

    void writeZeros(const long long &bytesToRead);
};

#endif /* _RSAENCRYPTOR_H */
```

```
#include "RSAencryptor.h"

RSAencryptor::RSAencryptor(bool const inUseHD, bool const outUseHD, unsigned int level)
: anEncryptor(inUseHD, outUseHD)
{
   /*
    *Set the base encryption level to level
    *create the random number generator used in generator
    *generate the needed keys
    */
   encryptionLevel = level;
   srand(time(NULL));
   generateKeys();
}//end RSAencryptor

RSAencryptor::~RSAencryptor()
{
   system("rm -f RSAtemp");
}//end destructor

void RSAencryptor::addPadding(string filename)
{
   /*
    *After doing the encryption, to help decryption pad the end of the file
    *with bytes to
    *pad: fielsize % limbsize = 0
    */
   string msg = "cp " + filename + " RSAtemp";
   system(msg.c_str());
   input->open(filename);
   ofstream o("RSAtemp", ios::app | ios::binary | ios::out);

   const unsigned char padding = sizeof(mp_limb_t) - (input->getLength() %
sizeof(mp_limb_t));

   for(unsigned int x =0; x < padding; x++)
   {o.write((char*)&padding, sizeof(padding));}
```

```
    input->close();
    o.close();
}//end void addPAdding()

void RSAencryptor::decrypt()
{
    input->open("RSAciphered");
    output->open("RSAtemp");
    mpz_class data,temp;
    long long bytesToRead = mpz_size(publicKey.primeMult.get_mpz_t()) * sizeof(mp_limb_t);

    for(unsigned int x =0; x < (input->getLength() / bytesToRead) -1; ++x)
    {
        input->read(data, bytesToRead);
        temp = decrypt(data);

       /*
        *Fills in zeros where needeed for the decryption
        *an attempt to alleviate some of the problem
        */

        for(unsigned int inner = 0;  inner < (mpz_size(publicKey.primeMult.get_mpz_t())
            - (mpz_size(temp.get_mpz_t()) + 1)); ++inner)
        {
            static const mp_limb_t limb =0;
            output->write((char*)&limb, sizeof(limb));
        }//end for inner < mpz_size(primemult) - (mpz_size(temp) + 1)

        output->write(temp);
    }//end for int x = <; x < input->getLength() / bytes to read -1

    input->read(data, bytesToRead);
    temp = decrypt(data);
    output->write(temp);

    input->close();
    output->close();
    removePadding();
}//end decrypt()
```

```
mpz_class RSAencryptor::decrypt(const mpz_class &data)
{
   mpz_class result =0;

   mpz_powm(result.get_mpz_t(), data.get_mpz_t(), privateKey.get_mpz_t(),
        publicKey.primeMult.get_mpz_t());

   return result;
}//end decrypt

void RSAencryptor::encrypt(string filename)
{
   /*
    *Idea behind public / private key is to have two people
    *therefore with no one else use myself as the second person
    */
   encrypt(this, filename);
}//end encrypt

void RSAencryptor::writeZeros(const long long &bytesToRead)
{
   const unsigned char z =0;
   for(int x =0; x < bytesToRead; ++x)
      output->write((char*)&z, sizeof(z));
}//end writeZeros

void RSAencryptor::encrypt(RSAencryptor* Bob, string filename)
{
   /*
    *In classic nomenclature Bob recieves a message and Alice sends
    *Therefore we assume we are Alice, and Bob is to recieve this message
    */
   addPadding(filename); // ensure our file is an whole number of limbs
   input->open("RSAtemp");
   output->open("RSAciphered");
   mpz_class data;

   /*Calculate the number of bytesToRead and the amount leftover based
```

```
    */
   const long long bytesToRead = (mpz_size(Bob->publicKey.primeMult.get_mpz_t()) -1) *
sizeof(mp_limb_t);
   const long long leftOver = input->getLength() % bytesToRead;

   mpz_class temp;

   for(unsigned long long x =0; x < (input->getLength()) / bytesToRead; ++x)
   {
      input->read(data, bytesToRead);
      temp = encrypt(data,*Bob);

      /*
       *An attempt to lessen the leading zero problem
       */

      if(!cmp(temp, 0))
      {
         writeZeros(bytesToRead);
      }//end if !cmp(temp,0
      else
      {
         output->write(encrypt(data, *Bob));
      }//end else

   }//end for(long long x < input->getLength() / bytesToRead)

   if(leftOver > 0)
   {
      input->read(data, leftOver);
      temp = encrypt(data,*Bob);

      if(!cmp(temp, 0))
      {
         writeZeros(leftOver);
      }//end if !cmp(temp,0
      else
      {
         output->write(temp);
```

```
      }//end else
   }//end if leftOver > 0

   input->close();
   output->close();
}//end encrypt(*bob)

mpz_class RSAencryptor::encrypt(const mpz_class &data, const RSAencryptor &Bob)
{
   mpz_class result =0;

   mpz_powm(result.get_mpz_t(), data.get_mpz_t(), Bob.publicKey.encryptExp.get_mpz_t(),
        Bob.publicKey.primeMult.get_mpz_t());
   return result;
}//end mpz_class RSAencryptor::encrypt(const mpz_class &data, RSAencryptor *Bob)

void RSAencryptor::generateKeys()
{
   //Declare variables
   mpz_class primeP, primeQ, theta, temp;
   gmp_randclass grc(gmp_randinit_default);

   ////////////generate two large and distinct primes about the same size//////////
   mpz_ui_pow_ui (primeP.get_mpz_t(),2,(encryptionLevel/2)-2);
   mpz_ui_pow_ui (primeQ.get_mpz_t(),2,(encryptionLevel/2));

   do
   {
      makeRandPrime(primeP);
      makeRandPrime(primeQ);
   }while( cmp(primeP, primeQ) == 0);// do while primeP = primeQ

   ////////end generate two large and distinct primes about the same size//////////

   //////// set publicKey.primeMult = pq and theta = (p-1)(q-1) ///////////
   publicKey.primeMult = primeP * primeQ;
   primeP = primeP - 1;
   primeQ = primeQ - 1;
   theta = primeP * primeQ;
```

```
    //////end set publicKey.primeMult = pq and theta = (p-1)(q-1) ///////////

    ///// 1 < encryptExp < theta && gcd(e, theta) =1)
    do
    {
     temp = theta -2;//to ensure that encryptExp > 1
     publicKey.encryptExp = grc.get_z_range(theta);
     publicKey.encryptExp = publicKey.encryptExp + 2;

     mpz_gcd(temp.get_mpz_t(), theta.get_mpz_t(), publicKey.encryptExp.get_mpz_t());
    }while( cmp(temp, 1) !=0 );//until temp ==1

    //this assigns the privateKey
    mpz_gcdext(temp.get_mpz_t(), privateKey.get_mpz_t(), NULL,
          publicKey.encryptExp.get_mpz_t(), theta.get_mpz_t());
}//end void RSAencryptor::generateKeys()

void RSAencryptor::makeRandPrime(mpz_class &prime)
{
    int randNum = (rand() % 100) + 1;
    for(int x =0; x < randNum; ++x)
    {mpz_nextprime(prime.get_mpz_t(), prime.get_mpz_t());}
}//end makeRandPrime(&mpz_class)

void RSAencryptor::removePadding()
{//RSAtemp -> RSAdeciphered
    input->open("RSAtemp");
    output->open("RSAdeciphered");

    unsigned char temp[sizeof(mp_limb_t)];

    for(unsigned long long x = 0; x < input->getLength() - sizeof(mp_limb_t); ++x)
    {
      input->read((char*)&temp[0], sizeof(temp[0]));
      output->write((char*)&temp[0], sizeof(temp[0]));
    }//end for( x < input->getLength() - sizeof(mp_limb_t))

    input->read((char*)&temp[0], sizeof(mp_limb_t));
    output->write((char*)&temp[0], sizeof(mp_limb_t) - temp[sizeof(mp_limb_t)-1]);
```

```
    input->close();
    output->close();
}//end removePAdding
```

```
#ifndef _RIJNDAEL_H
#define	_RIJNDAEL_H

//Taken from http://en.wikipedia.org/wiki/Rijndael_S-box October 28, 2007

const static unsigned char sbox[256] =
{
//0     1    2    3    4    5    6    7    8    9    A    B    C    D    E    F
0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab,
0x76, //0
0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72,
0xc0, //1
0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31,
0x15, //2
0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75,
//3
0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f,
0x84, //4
0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf,
//5
0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f,
0xa8, //6
0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3,
0xd2, //7
0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19,
0x73, //8
0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b,
0xdb, //9
0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4,
0x79, //A
0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae,
0x08, //B
0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b,
0x8a, //C
0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d,
0x9e, //D
0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28,
0xdf, //E
```

```
0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16
//F
};

const static unsigned char pi[16] =
{
 0x24, 0x3f, 0x6a, 0x88, 0x85, 0xa3, 0x08, 0xd3,
 0x13, 0x19, 0x8a, 0x2e, 0x03, 0x7d, 0x73, 0x44
};

#endif /* _RIJNDAEL_H */
```

```
#ifndef _SCREAMENCRYPTOR_H
#define        _SCREAMENCRYPTOR_H

#include "anEncryptor.h"
#include "SCREAMconsts.h"

class SCREAMencryptor : public anEncryptor
{
public:
   SCREAMencryptor(bool const inUseHD, bool const outUseHd);
   virtual ~SCREAMencryptor(){}
   virtual void encrypt(string filename);
   virtual void decrypt();

   unsigned long long int getTableSetup(){return tableSetup;}
private:
   unsigned char GLBLX[16], GLBLY[16], GLBLZ[16], tableW[256], indexW;
   unsigned long long int tableSetup;

   void initialize();
   void createKey(unsigned char* tempW);
   void createNonce(const unsigned char* tempW);
   void XORNbyN(unsigned char const *left, unsigned char const *right,
        const unsigned int &N, unsigned char result[]);
   void XOROneF(const unsigned char* left, const unsigned char* right, const int &num,
     unsigned char result[]);
   void functionF(const unsigned char* array16, unsigned char result[]);
   unsigned char* getOutputBytes(unsigned long long num);
   void functionT0(const unsigned char &byte, unsigned char result[]);
   void functionT1(const unsigned char &byte, unsigned char result[]);
   void functionTmain(const unsigned char &byte, const int &num, unsigned char result[]);
   unsigned char getMatrix(const int &row, const int &col, const unsigned char &byte,
        const unsigned char &ver);

   void nonceHelper(const unsigned char* left, const unsigned char* right,
     unsigned char result[]);

   unsigned char sbox1(unsigned char lookup){return sbox[lookup];}
```

```
    unsigned char sbox2(unsigned char lookup){return sbox[lookup ^ 21];}
};
#endif /* _SCREAMENCRYPTOR_H */
```

```
#include "SCREAMencryptor.h"

SCREAMencryptor::SCREAMencryptor(const bool inUD, const bool outUD)
: anEncryptor(inUD, outUD)
{
   indexW =0;
   for(int x =0; x < 16; ++x)
   {
      GLBLX[x] = GLBLY[x] = GLBLZ[x] = 0;
      for(int y =0; y < 16; ++y)
      {
         tableW[16*x + y] =0;
      }
   }

   unsigned char tempW[256];
   createKey(tempW);
   createNonce(tempW);
}

void SCREAMencryptor::functionF(const unsigned char *arrayIn, unsigned char result[])
{
   unsigned char temp[16], holder[4], temp4[4];
   for(int x =0; x < 16; ++x)
      {result[x] = arrayIn[x];}

   // First half round ////////////////////////////////////////
   for(int x =0; x <4; ++x)
   {
      functionT0(result[x*4], holder);
      functionT1(result[((x*4)+13)%16], temp4);
      XORNbyN(holder, temp4, 4, holder);
      for(int y =0; y < 4; ++y)
      {
         temp[(x*4)+y]=holder[y];
      }//end for y < 4
   }//end x < 4
```

```
for(int x =0; x < 4; ++x)
{
   XORNbyN((const unsigned char*)&temp[(x*4)+2],
        (const unsigned char*)&result[(x*4)+2],2, holder);
   for(int y =0; y < 2; ++y)
   {
      temp[(x*4)+2+y]=holder[y];
   }//end y < 2
}//end x < 4
////////////////////////////////////////////////////////////////////////////

//swapping
for(int x=0; x < 4; ++x)
{
   swap(temp[(x*4) + 2], temp[(x*4)]);
   swap(temp[(x*4) + 3], temp[(x*4) + 1]);
}
////////////////////////////////////////////////////////////////////////////

// second half round
for(int x =0; x <4; ++x)
{
   functionT0(temp[((x*4)+8)%16], holder);
   functionT1(temp[((x*4)+9)%16], temp4);
   XORNbyN(holder, temp4, 4, holder);
   for(int y =0; y < 4; ++y)
   {
      result[(x*4)+y]=holder[y];
   }//end for y < 4
}//end x < 4

for(int x =0; x < 4; ++x)
{
   XORNbyN((const unsigned char*)&temp[(x*4)+2],
        (const unsigned char*)&result[(x*4)+2],2, holder);
   for(int y =0; y < 2; ++y)
   {
      result[(x*4)+2+y]=holder[y];
   }//end y < 2
```

```
    }//end x < 4
    ////////////////////////////////////////////////////////////////////////
}//end unsigned char* SCREAMencryptor::functionF(unsigned char *array16)

void SCREAMencryptor::createKey(unsigned char *tempW)
{
    unsigned char seed[16], a[16], b[16];

    for(unsigned char x =0; x < 16; ++x)
    {seed[x] = x;}
////////////////////////////////////////
    for(int x =0; x < 16; ++x)
    {a[x] = seed[x];}

    XORNbyN(a, pi, 16, b);
    functionF(b, b);
    for(int x =0; x < 16; ++x)
    {
        for(int y =0; y < 4; ++y)
        {functionF(a, a);}//end y < 4

        XORNbyN(a, b, 16, a);

        for(int y =0; y < 16; ++y)
        {tempW[(16 * x) + y] = a[y];}//end y < 16
    }//end x < 16
}//end void createKey(unsigned char* tempW)

void SCREAMencryptor::nonceHelper(const unsigned char* left, const unsigned char* right,
        unsigned char result[])
{
    XORNbyN(left, right, 16, result);
    functionF(result, result);
    functionF(result, result);
}//end nonceHelper

void SCREAMencryptor::createNonce(const unsigned char* tempW)
{
    unsigned char a[16], b[16], nonce[16];
```

```
    for(unsigned char x = 0; x < 16; ++x)
    {nonce[x] = x;}

    nonceHelper((const unsigned char*)&nonce[0], &tempW[16], GLBLZ);

    nonceHelper(GLBLZ, &tempW[16*3], GLBLY);

    nonceHelper(GLBLY, &tempW[16*5], a);

    XORNbyN(a, &tempW[16*3], 16, GLBLX);

    for(int x=0; x< 16; ++x)
    {b[x] =GLBLX[x];}

    for(int x =0; x < 8; ++x)
    {
      //note can now use nonce as a temp variable
      XORoneF(&tempW[2*x*16], b, 16, b);
      //XORNbyN((const unsigned char *)&tempW[2 * x * 16], b, 16, b);
      //functionF(b,b);

      /*
       *Below is not functioned since this is only place where use it,
       *and the result would be a large function call every time
       */

      XORNbyN((const unsigned char*)&tempW[2*x*16], a, 16, nonce);
      for(int y =0; y < 16; ++y)
      {tableW[(2*x*16)+y]=nonce[y];}

      XORNbyN((const unsigned char*)&tempW[(2*x*16) + 16], b, 16, nonce);
      for(int y =0; y < 16; ++y)
      {tableW[(2*x*16)+16+y]=nonce[y];}
    }//end for x < 8
}//end void createNonce(unsigned char* tempW)

void SCREAMencryptor::XORoneF(const unsigned char* left, const unsigned char* right, const
int &num,
```

```
    unsigned char result[])
{
  XORNbyN(left, right, num, result);
  functionF(result, result);
}

void SCREAMencryptor::XORNbyN(unsigned char const *left, unsigned char const *right,
    const unsigned int &N, unsigned char result[])
{
  for(unsigned int x =0; x < N; ++x)
  {result[x] = left[x] ^ right[x];}
}//end unsigned char* XORNbyN(unsigned char const *left, unsigned char const *right, int N)

void SCREAMencryptor::functionT0(const unsigned char &byte, unsigned char result[])
{
  return functionTmain(byte, 0, (unsigned char *)result);
}//end unsigned char* SCREAMencryptor::functionT0(unsigned char byte)

void SCREAMencryptor::functionT1(const unsigned char &byte, unsigned char result[])
{
  return functionTmain(byte, 1, result);
}//end unsigned char* SCREAMencryptor::functionT1(unsigned char byte)

void SCREAMencryptor::functionTmain(const unsigned char &byte, const int &num,
    unsigned char result[])
{
  result[0] = getMatrix(0,num, byte,(unsigned char)1) * sbox1(byte);
  result[1] = getMatrix(1,num, byte,(unsigned char)1) * sbox1(byte);
  result[2] = getMatrix(0,num, byte,(unsigned char)2) * sbox2(byte);
  result[3] = getMatrix(1,num, byte,(unsigned char)2) * sbox2(byte);
}//end unsigned char* SCREAMencryptor::functionTmain(unsigned char byte, int num)

unsigned char SCREAMencryptor::getMatrix(const int &row, const int &col,
    const unsigned char &byte, const unsigned char &ver)
{
  if(row == col)
    return 1;

  return byte + ver -1;
```

```
}//end unsigned char SCREAMencryptor::getMatrix1(int row, int col, unsigned char byte)

unsigned char* SCREAMencryptor::getOutputBytes(unsigned long long num)
{
   num += (num % 16); //makes sure that a whole GLBLX size can fit
   unsigned long long index =0;
   unsigned char temp[16];
   unsigned char* result = new unsigned char[num];
   unsigned char* vars= new unsigned char[num];

   for(unsigned int x =0; x < (num / 16); ++x)
   {
      for(int y =0; y < 16; ++y)
      {
         XORNbyN(GLBLX, GLBLY, 16, GLBLX);
         functionF(GLBLX, GLBLX);
         XORNbyN(GLBLX, GLBLZ, 16, GLBLX);

         XORNbyN(GLBLX, (unsigned char* const)&tableW[((x*16)+y) % 240], 16, temp);
         for(int z =0; z < 16; ++z)
         {result[index + z] = temp[z];}//end for z < 16

         if(y ==0 || y %4 ==2)
         {
            for(int z =0; z < 8; ++z)
            {swap(GLBLY[z], GLBLY[z+8]);}
         }//end if(y ==0 || y %4 ==2)
         else if(y%4 ==1)
         {
            for(int z=0; z < 4; ++z)
            {
               swap(GLBLY[z], GLBLY[z+4]);
               swap(GLBLY[z+8], GLBLY[z+12]);
            }
         }//end if y%4 ==1
         else if ( y < 15)
         {
            for(int z=0; z < 8; ++z)
            {
```

```
                swap(GLBLY[z], GLBLY[(z+5) % 8]);
                swap(GLBLY[z+8], GLBLY[(z+13) % 16]);
              }//end z < 8
            }//end if y < 15
          }//end for y < 16

          XORoneF(GLBLY, GLBLZ, 16, GLBLY);

          XORoneF(GLBLZ, GLBLY, 16, GLBLZ);

          functionF((unsigned char *)&tableW[indexW], temp);
          for(int z=0; z < 16; ++z)
          {tableW[indexW + z] = temp[z];}

          indexW = (indexW + 16) % 256;
          index += 16;
        }//end for x < num
        delete [] vars;
        return result;
    }//end getOutputBytes

    void SCREAMencryptor::encrypt(string filename)
    {
        input->open(filename);
        output->open("SCREAMciphered");
        unsigned char *vars = getOutputBytes(input->getLength());

        unsigned char read;
        for(unsigned long long x =0; x < input->getLength(); ++x)
        {
            input->read((char*)&read, 1);
            read ^= vars[x];
            output->write((char*)&read,1);
        }
        input->close();
        output->close();

        delete []vars;
    }//end encrypt
```

```
void SCREAMencryptor::decrypt()
{
   input->open("SCREAMciphered");
   output->open("SCREAMdeciphered");
   unsigned char* vars = getOutputBytes(input->getLength());

   unsigned char read;
   for(unsigned long long x =0; x < input->getLength(); ++x)
   {
      input->read((char*)&read, 1);
      read ^= vars[x];
      output->write((char*)&read,1);
   }
   input->close();
   output->close();

   delete []vars;
}//end decrypt
```

```
#ifndef _WRITERS_H
#define	_WRITERS_H

#include <fstream>
#include <gmpxx.h>
using namespace std;

class WRITERS : protected ofstream
{
public:
   /*
    *all WRITERS at some point need to access files, so inhereting from ofstream
    *works well here (Thanks Gary!). The function names are identical versions of
    *ofstream.  ofstream gets called via ofstream:: as needed
    *Therefore the virtualness of the below functions.
    */
   virtual void close() = 0;
   virtual void open(const string &t) =0;
   virtual void write(const char *p, const int &n)=0;
   /*
    * since virtual can divide the mpz_class into bytes and use the virtual
    * version of write to write each byte
    */
   virtual void write(const mpz_class &writeme);

   //setSize has no bearing on diskReader, but allows for space to be made in the
   //memReader.
   virtual void setSize(const long long &mysize){ }//does nothing unless memReader
};

#endif /* _WRITERS_H */
```

```
#include "WRITERS.h"

void WRITERS::write(const mpz_class &writeme)
{
   /*
    *This should call the current write(char*, int)
    *This imverts due to the way mpz handles numbers
    */
   mp_limb_t limb;
   for(int x = mpz_size(writeme.get_mpz_t()) -1; x >=0; --x)
   {
      limb = mpz_getlimbn(writeme.get_mpz_t(), x);
      write((char *)&limb, sizeof(limb));
   }//end for x >= 0
}//end write(mpz_class)
```

Program Documentation

Program Execution

Primary executable code, sp_in_c, has two options for running, one executes the code and the second presents an error message describing correct usage. To execute the code in a Linux environment the following parameters must be passed:

- filename to encrypt
- Type of encryption (capitalization counts)
    - IDEA
    - RSA
    - SCREAM
- Decide whether to use diskReader/diskWriter or not. This has a bearing on speed, if the memory is to used, then do not use diskReader/diskWriter, for larger files use diskReader/diskWriter
    - yes
    - no
- Finally whether to encrypt or decrypt with the above information
    - encrypt
    - decrypt

It is important to note from the above, that there is no error checking on the file name so this must be type in correctly. For the others they would fail the if checks, so they would simply not execute. If 4 parameters are not passed to sp_in_c, then an help message appears stating what data must be passed and in what order.

Imported Libraries

The following quote and link were copied from http://gmplib.org/ on December 10th, 2007. As I am not charging for my products it is therefore free and allowed to be used. "GMP is distributed under the GNU LGPL. This license makes the library free to use, share, and improve, and allows you to pass on the result. The license gives freedoms, but also sets firm restrictions on the use with non-free programs."

Known Bugs

It is known that RSA does not correctly handle some files. It has been determined that RSA fails when there are leading zeros. The Gnu Multiple Precision Library (gmp), handles the large numbers as a series of a data type called limb. Limbs are set to be the same bit length as the word size of the computer. With the current definition of a limb and the size of an int this means it is possible (not probable) to have an error with 32 consecutive off bits. This will only cause a problem if they are the first set in a given mpz (the data structure to hold the growing number of bits, Multiple Precision from mathematical set Z).

Results

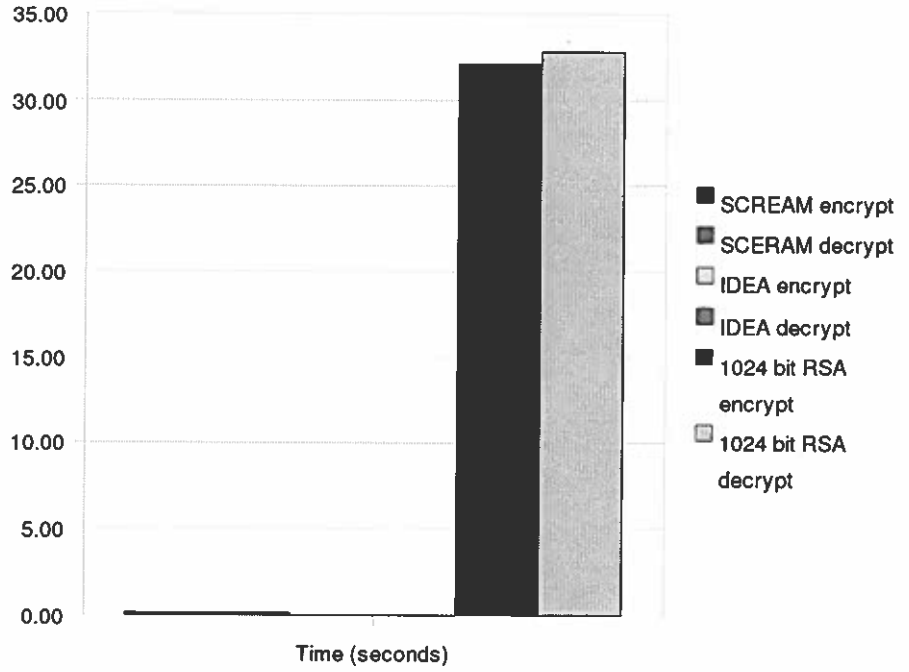
Time and Security

The security of the different ciphers is presented to me in different measurements. Therefore the absolute security involved has to be inferred. The newest IDEA algorithm was evaluated based on computational complexity, the SCREAM cipher was analyzed with how many bytes until the cipher becomes breakable, and RSA is evaluated by MIPS-Years. While I as unable to find and exact measurement, it is logical to think that computational complexity and MIPS-Years would be proportional. The table below shows these numbers based of the following documents: "FOX: a New Family of Block Ciphers" (FOX was the name of IDEA before changed to IDEA), "RSA Laboratories – A Cost-Based Security Analysis of Symmetric and Asymmetric Key Lengths", and "Scream: A Software-efficient Stream Cipher".

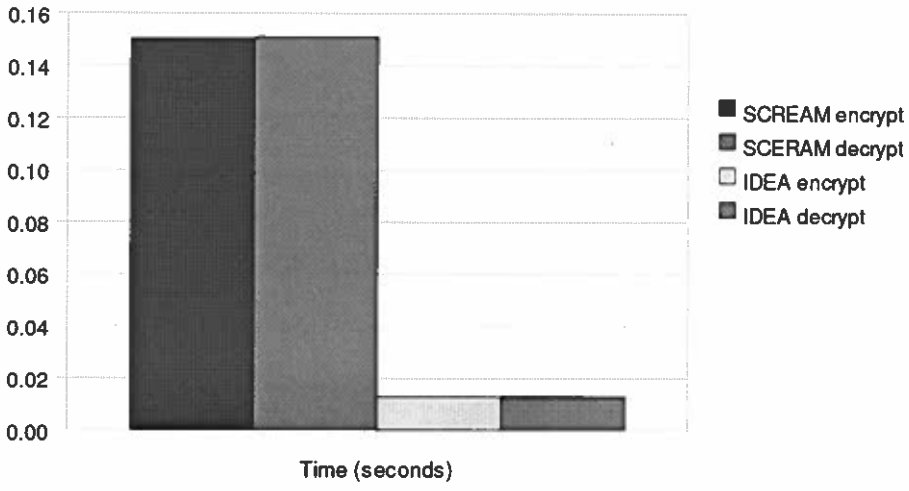
Algorithm	Security
IDEA	Computational Complexity of 2^{128}
SCREAM	Each nonce / key pair secure up to about 2^{44} bytes
RSA 1024 bit	5.60E+010 MIPS-Years

The timing side of security was a bit of a challenge to fairly evaluate. In an effort to isolate the algorithm itself a few test runs were executed. All time trials were ran Xubuntu 7.10 running on VMware server 1.0.4 in the AH 203 lab. Using optimized code, I ran the total program 1000 and got the user time from Linux's time function. I then edited my code to only

include any time required to set up tables (RSA and SCREAM only), and a third time only including uses of ifstream and ofstream (directly or indirectly). After these numbers were given, I subtracted the table and IO time and the remaining time is given below in tables.



Difference between SCREAM/IDEA and RSA



The difference between SCREAM and IDEA

What I Learned

Throughout the design and implementation of this senior project, the advantage of planning continued to be present. As my code was done in an object oriented fashion, doing as much designing first definitely was a needed decision. The real life aspects of adding features and major changes midway through also came into play.

I had originally thought of doing the code in Java, as that would allow the most portability and hopefully the most readable code. This however not to work out as well as I had hoped, since getting bit access seemed to be much more difficult in Java than in C++. The necessity of 2 readers and 2 writers also became an unexpected bump. While these were overcome, they applied a different perspective to projects – the element of not knowing.

The other major thing that I learned was that the one helping with an assignment may not always have the answer. Where academically most of the time instructor has seen the issues that students come up with, this project was new to both Gary and I. Therefore there was no "usual problems" to base things off of. This helped me to learn how to find my own answers, a skill I know I will use in the real world. More importantly it taught me how to bounce ideas off of another professional who does not know the entire scope, therefore helping my communication in explaining what is important to the problem.

Works cited

Assche, Gilles Van. Quantum Cryptography and Secret-Key Distillation Cambridge: Cambridge University Press, 2006

"Definition: nonce" http://www.atis.org/. 2001. ATIS Committee T1A1. 7 December 2007 < http://www.atis.org/tg2k/_nonce.html>

Halevi, Shai, Don Coppersmith and Charanjit Jutla. "Scream: A Software-efficient Stream Cipher". Yorktown Heights, New York: IBM T. J. Watson Research Center, June 5, 2002.

Junod, Pascal and Vaudenay, Serge. "FOX : a New Family of Block Ciphers". 2004.

Lai, Xuejia. "答复: Question on IDEA." Email to Xuejia Lai. 17 November 2007

Menezes, Alfred J. et. al. Handbook of Applied Cryptography New York: CRC press, 1997

"RSA Laboratories – A Cost-Based Security Analysis of Symmetric and Asymmetric Key Lengths" www.rsa.com. 2007. RSA Laboratories. 09 December 2007 <http://www.rsa.com/rsalabs/node.asp?id=2088>

"RSA Laboratories - 2.1.4 What is a block cipher?" www.rsa.com. 2007. RSA Laboratories. 17 November 2007 <http://www.rsa.com/rsalabs/node.asp?id=2168>

"RSA Laboratories - 3.6.7 What are some other block ciphers?" www.rsa.com. 2007. RSA Laboratories. 17 November 2007 <http://www.rsa.com/rsalabs/node.asp?id=2254>

Rivest, Ronald L. "Re: About RSA." Email to Ronald L. Rivest. 18 November 2007

United States Department of Commerce. Recommendation for Block Cipher Modes of

Operation. NIST Special Publication 800-38A 2001 Edition December 2001

Weak_Keys_0206. Zurich, Switzerland: Mediacrypt 2006